

EV-AUT-03 — Portafolio de Evidencias Individuales

Estudiante: Jhinson Stalyn Aucatoma Celorio
Curso: 7mo Software “A”
Equipo PFC: AGLS
Código del instrumento: EV-AUT-03

Integrantes PFC:

- Jhinson Stalyn Aucatoma Celorio
- Jeremy Ruperto Gaibor Rodríguez
- Andy Paul Sánchez Pilaloe
- José Alejandro Lozano Morales

URL del repositorio del portafolio:

https://github.com/JhinsonAucatoma/ev_sum_04_portafolio

URL del repositorio del PFC:

<https://github.com/JoseLozanoMorales/PFC-AppsDistribuidas>

1. Evidencias de aprendizaje del periodo

1.1. Evidencia 1 — Código propio: criterio de fragmentación horizontal

Unidad del curso: Unidad 3. Bases de Datos Distribuidas (Tema 2: Diseño y Fragmentación de Bases de Datos Distribuidas).

Artefacto:

- Archivo: `sql/03_fragmentacion_horizontal.sql`
- Commit: 1640a76 (hash completo: 1640a76d196694c66c4961684013548ba3780fe6)
- Repositorio: <https://github.com/JhinsonAucatoma/practica-fragmentacion>

Listing 1: Fragmento de `03_fragmentacion_horizontal.sql`: esquema del fragmento horizontal de pedidos.

```
1 CREATE TABLE pedidos_fragmento (  
2     pedido_id INTEGER PRIMARY KEY,  
3     cliente_id INTEGER NOT NULL,  
4     producto_id INTEGER NOT NULL,  
5     fecha DATE NOT NULL,  
6     monto NUMERIC(8,2) NOT NULL,  
7     sede VARCHAR(20) NOT NULL  
8 );
```

Explicación del aprendizaje. Al diseñar el esquema de fragmentación horizontal de la tabla `pedidos` (Listado 1) evalué dos criterios de corte posibles: por `sede` o por `producto_id`. Descarté `producto_id` porque las consultas típicas del escenario (una cafetería con tres sedes) son del tipo “ventas de esta sede”, no “ventas de este producto”; fragmentar por producto no habría acercado los datos a quien los consulta. Entonces opté por `sede` porque ese criterio hace que cada fragmento resida físicamente cerca de los usuarios que más lo consultan, lo que en *Principles of Distributed*

Database Systems se plantea como uno de los objetivos centrales del diseño de fragmentación: minimizar el costo de las consultas más frecuentes acercando los datos a su patrón de acceso [1]. Lo que aprendí es que una decisión de fragmentación no se valida solo verificando que sea correcta en abstracto (completitud, reconstrucción, disjunción), sino evaluando si el criterio de corte refleja cómo se va a consultar el dato en la práctica; dos esquemas igualmente correctos pueden tener rendimientos muy distintos según ese ajuste.

1.2. Evidencia 2 — Análisis técnico: verificación cuantitativa de consistencia

Unidad del curso: Unidad 3. Bases de Datos Distribuidas (Tema 2: Diseño y Fragmentación de Bases de Datos Distribuidas).

Artefacto:

- Archivo: `sql/07_verificacion.sql`
- Commit: 1640a76 (hash completo: 1640a76d196694c66c4961684013548ba3780fe6)
- Repositorio: <https://github.com/JhinsonAucatoma/practica-fragmentacion>

Listing 2: Consultas de `07_verificacion.sql` usadas para medir completitud, reconstrucción y disjunción.

```

1  -- 1) COMPLETITUD
2  SELECT COUNT(*) AS filas_globales FROM pedidos_global;
3  SELECT COUNT(*) AS filas_centralizadas FROM pedidos;
4
5  -- 2) RECONSTRUCCION
6  SELECT sede, SUM(monto) AS total_global
7  FROM pedidos_global GROUP BY sede ORDER BY sede;
8  SELECT sede, SUM(monto) AS total_centralizado
9  FROM pedidos GROUP BY sede ORDER BY sede;
10
11 -- 3) DISJUNCION
12 SELECT pedido_id, COUNT(*) AS veces
13 FROM pedidos_global
14 GROUP BY pedido_id
15 HAVING COUNT(*) > 1;

```

Cuadro 1: Resultados obtenidos al ejecutar las consultas de verificación sobre el conjunto de prueba (6 clientes, 8 pedidos repartidos en 3 sedes).

Condición	Métrica medida	Valor obtenido	¿Coincide?
Completitud	filas_globales vs. filas_centralizadas	8 vs. 8	Sí
Reconstrucción	Suma de monto por sede (global vs. centralizado)	Babahoyo 4.25, Campus 4.25, Ventanas 2.25 (ambas consultas)	Sí
Disjunción	filas con <code>pedido_id</code> repetido	0 filas	Sí (0 esperado)

Protocolo de medición. Tras cargar el conjunto de prueba (6 clientes y 8 pedidos, repartidos de forma desigual entre Campus, Babahoyo y Ventanas) y construir la vista `pedidos_global` mediante `postgres_fdw` (unión de los tres fragmentos horizontales), ejecuté por `psql` dentro del contenedor `pg-campus` las tres consultas del Listado 2 y comparé cada resultado contra la tabla `pedidos` de ese mismo nodo, que conservé sin fragmentar como referencia central (Tabla 1).

Interpretación. Los tres resultados confirman, con datos medidos por mí, las tres condiciones de corrección que exige toda fragmentación correcta según la literatura del curso: completitud

(ningún dato se perdió: 8 filas en ambas consultas), reconstrucción (UNION ALL reproduce exactamente los mismos totales por sede que la tabla centralizada) y disjunción (ningún `pedido_id` aparece en más de un fragmento) [1]. Sin embargo, esta verificación tiene una limitación que no debo presentar como concluyente: se ejecutó sobre un conjunto sintético de solo 8 filas, en un único punto en el tiempo y sin escritura concurrente entre nodos. No comprueba, por ejemplo, qué pasaría si dos sedes insertaran pedidos simultáneamente mientras otro cliente consulta `pedidos_global` un escenario donde los compromisos entre disponibilidad, latencia y consistencia que describe el modelo PACELC sí serían relevantes y podrían mostrar inconsistencias transitorias que esta prueba estática, de un solo escritor, no puede detectar [2]. Lo que aprendí es que verificar completitud/reconstrucción/disjunción sobre datos estáticos prueba la corrección del *esquema* de fragmentación, pero no sustituye una prueba de consistencia bajo concurrencia real.

1.3. Evidencia 3 — Reflexión documentada: de un fallo total a una degradación controlada

Unidad del curso: Unidad 2. Aplicaciones Distribuidas: fragmentación horizontal, transparencia de ubicación y autonomía local (caso Banco del Austro).

Artefacto:

- Archivo: `service/ConsultaDistribuidaService.java`
- Método: `listarTodosLosClientes`
- Commit: `e8e57ee` (hash completo: `e8e57eed14cefa94762c28db9d5884c0b2bbcaf2`)

Repositorio: https://github.com/JhinsonAucatoma/TA_IND_02_Analisis_Consistencia_Replicacion_BDD

El hecho concreto. Al detener el contenedor `banco-quito` para probar la tolerancia a fallos, la versión original de `listarTodosLosClientes()` (que simplemente concatenaba las consultas de los dos `JdbcTemplate` (`jdbcCuenca` y `jdbcQuito`) sin ningún manejo de errores) respondió con un error 500 Internal Server Error, incluso para los clientes de Cuenca, cuyo nodo seguía activo. La versión corregida quedó así:

Listing 3: Fragmento de `listarTodosLosClientes` con manejo de fallos por nodo.

```
1 try { union.addAll(jdbcCuenca.queryForList(sql)); }
2 catch (Exception e) { nodosCaidos.add("CUENCA"); }
3
4 try { union.addAll(jdbcQuito.queryForList(sql)); }
5 catch (Exception e) { nodosCaidos.add("QUITO"); }
6
7 Map<String, Object> resultado = new HashMap<>();
8 resultado.put("clientes", union);
9 resultado.put("totalDisponibles", union.size());
10 if (!nodosCaidos.isEmpty()) {
11     resultado.put("advertencia",
12         "Los siguientes nodos no respondieron y se excluyeron de la
13         union: " + nodosCaidos);
14     resultado.put("nodosCaidos", nodosCaidos);
15 }
```

Reflexión. El supuesto que resultó falso fue asumir, de forma implícita, que todos los nodos consultados en una operación distribuida iban a responder siempre. En el código original ese supuesto ni siquiera estaba escrito (simplemente no había ningún `try/catch`) lo cual es precisamente lo peligroso: un supuesto no declarado es más difícil de cuestionar que uno explícito. La caída de un único nodo (Quito) bastó para tumbar una respuesta que, en un 60 % de los casos (los clientes de Cuenca), sí se podía entregar. Al envolver cada consulta por nodo en su propio `try/catch` (Listado 3), cambié la decisión de diseño: en lugar de exigir que *todos* los nodos respondan para dar cualquier respuesta, el sistema ahora entrega lo que sí tiene disponible

junto con una advertencia explícita de qué nodos faltaron. Es una versión artesanal del patrón *Circuit Breaker* que describe Nygard para sistemas en producción: preferir una degradación visible y controlada antes que una falla total o, peor, una respuesta silenciosamente incompleta [3]. Lo que me llevo de esto, más allá de este proyecto puntual, es que en cualquier operación que dependa de múltiples nodos remotos debo preguntarme explícitamente “¿qué debe devolver este endpoint si uno de los nodos no responde?” *antes* de escribir el código de integración, no después de que una prueba e caída lo revele por accidente.

2. Contribución individual al PFC

Repositorio del PFC: <https://github.com/JoseLozanoMorales/PFC-AppsDistribuidas>

Captura del historial filtrado por autoría. La Figura 1 muestra el historial de commits del repositorio filtrado por mi usuario (*JhinsonAucatoma*) con el rango “All time”, obtenida desde la pestaña *Commits* de GitHub. Se reprodujo con `git log -author="jaucatomac@uteq.edu.ec"` desde una clonación local, confirmando que el historial completo (no un clon parcial) y una única identidad de autor (*JhinsonAucatoma* <jaucatomac@uteq.edu.ec>, verificada con `git log -all -format="%an <%ae>" | sort -u`) sustentan esta tabla.

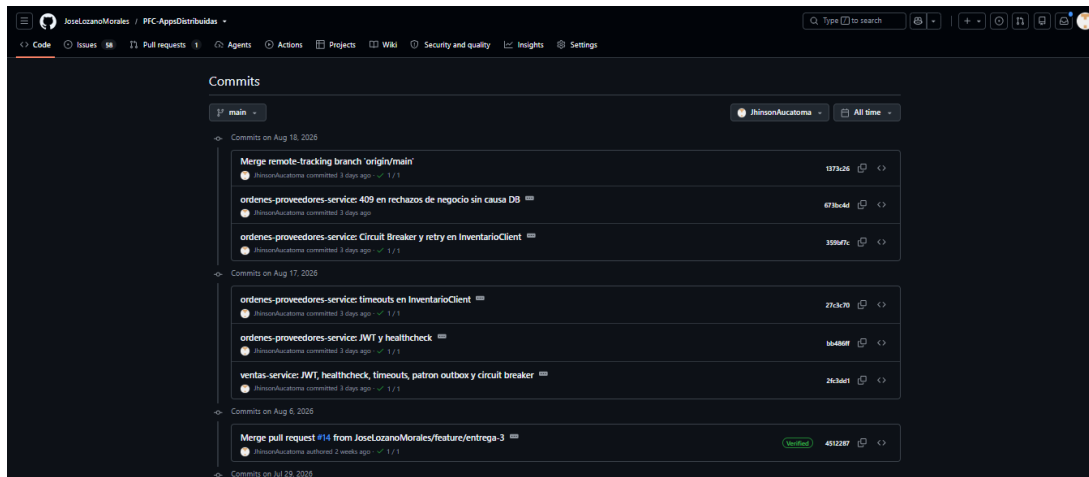


Figura 1: Historial de commits del repositorio del PFC filtrado por autoría (*JhinsonAucatoma*, “All time”).

Distribución temporal. Los aportes no se concentran en un único día ni en las horas previas a una entrega: se distribuyen en siete fechas distintas a lo largo del periodo (25, 26, 27 y 29 de julio; 6, 17 y 18 de agosto de 2026), con trabajo sustantivo en cada una — nuevos microservicios, manejo de errores, seguridad JWT, patrón outbox, circuit breaker/retry y la redacción de secciones del informe LaTeX del PFC.

Cuadro 2: Aportes individuales verificados en el repositorio del PFC (filtrados por autoría, `git log -numstat`).

Hash	Fecha	Mensaje	Archivos afectados
1373c26	2026-08-18	Merge remote-tracking branch 'origin/main'	(merge, sin cambios propios)
673bc4d	2026-08-18	ordenes-proveedores-service: 409 en rechazos de negocio sin causa DB	ApiExceptionHandler.java, PgErrorMapper.java, OrdenCompraController.java y 4 más

Hash	Fecha	Mensaje	Archivos afectados
359bf7c	2026-08-18	ordenes-proveedores-service: Circuit Breaker y retry en InventarioClient	pom.xml, InventarioClient.java, application.properties
27c3c70	2026-08-17	ordenes-proveedores-service: timeouts en InventarioClient	InventarioClient.java, application.properties
bb486ff	2026-08-17	ordenes-proveedores-service: JWT y healthcheck	Dockerfile, pom.xml, JwtAuthFilter.java y 2 más
2fc3dd1	2026-08-17	ventas-service: JWT, healthcheck, timeouts, patrón outbox y circuit breaker	17 archivos: JwtAuthFilter.java, SecurityConfig.java, InventarioOutboxProcessor.java, FacturaOutboxRepository.java y 13 más (merge, sin cambios propios)
4512287	2026-08-06	Merge pull request #14 from JoseLozanoMorales/feature/entrega-3	
f8d8218	2026-07-29	Quitar salto de página	docs/entrega3/PFC3.tex
a383849	2026-07-29	Conclusión del informe	docs/entrega3/PFC3.tex
496ee8a	2026-07-29	Declaración de uso de Inteligencia Artificial	docs/entrega3/PFC3.tex
e468360	2026-07-29	fundamento teórico	docs/entrega3/PFC3.tex
fbdbf60	2026-07-29	Bibliografía mínima y 7 con DOI verificable	docs/entrega3/referenciasPFC.bib
7acce5e	2026-07-27	ordenar línea del docker compose que conecta al microservicio de inventario	docker-compose.yml
e7fa5b0	2026-07-27	configuración para establecer la conexión con el microservicio de inventario-service para reducir stock tras la venta	docker-compose.yml, InventarioClient.java, FacturaService.java, application.properties
0af9b00	2026-07-27	agregación para devolver numero y total en la respuesta de POST /api/facturas ya que solo devuelve facturaId	docker-compose.yml, FacturaController.java
0cc603d	2026-07-26	Adición del microservicio de ventas al docker-compose	docker-compose.yml
126ea84	2026-07-26	Adición del microservicio de Ventas que confirma la venta de un cliente	10 archivos: Dockerfile, VentasServiceApplication.java, ApiExceptionHandler.java y 7 más
0fd33c2	2026-07-25	Adición de los archivos para el levantamiento del contenedor del microservicio de ordenes-proveedores	docker-compose.yml, .dockerignore, Dockerfile, application.properties
092ee41	2026-07-25	Merge remote-tracking branch 'origin/Matster_Repositories'	(merge, sin cambios propios)
123d37a	2026-07-25	corrección del CallableStatement ya que no llamaba a los procedimientos, añadir los controler de Orden y Proveedor, el Repository de OrdenCompra y Proveedor y avance de conexión al microservicio de Inventario	InventarioClient.java, OrdenCompraController.java, ProveedorController.java y 4 más

Hash	Fecha	Mensaje	Archivos afectados
ea77642	2026-07-25	arreglo del app Spring Boot, también se agregé para el manejo de errores y services de orden de compra/proveedor	9 archivos: OrdenesProveedoresServiceApplication.java, ApiExceptionHandler.java, PgErrorMapper.java y 6 más
8b96eb8	2026-07-25	Se agregaron el model y repository de órdenes de proveedores y la creación del servicio de ventas	9 archivos: .idea/misc.xml, pom.xml, DetalleOrdenCompra.java y 6 más

3. Declaración de uso de IA generativa

Se utilizó inteligencia artificial (Claude, Anthropic) como apoyo en la elaboración de este portafolio, con dos propósitos concretos. Primero, para mejorar la redacción, dar coherencia y conectar correctamente las ideas en la Sección 1 (Evidencias 1, 2 y 3) y en la Sección 2 (Contribución individual al PFC), sobre contenido, datos y análisis que yo redacté primero en borrador. Segundo, como asistencia con los comandos de `git` usados para construir la Sección 2, ya que inicialmente no lograba identificar de forma correcta todos mis aportes en el historial del repositorio: los commits no aparecían agrupados bajo una única identidad de autor. La IA sugirió verificar todas las identidades de autor presentes en el historial con `git log -all -format="%an <%ae>| sort -u` y filtrar luego con `git log -author`, usando el correo institucional (`jaucatomac@uteq.edu.ec`) en lugar del nombre de usuario, lo cual permitió confirmar que el historial completo y una única identidad (`JhinsonAucatoma <jaucatomac@uteq.edu.ec>`) sustentaban correctamente mis aportes. Las interpretaciones, reflexiones y análisis presentados en este documento pertenecen al autor.

Referencias

- [1] M. T. Özsu y P. Valduriez, *Principles of Distributed Database Systems*, 4.^a ed. Cham, Switzerland: Springer, 2020. DOI: [10.1007/978-3-030-26253-2](https://doi.org/10.1007/978-3-030-26253-2).
- [2] D. Abadi, “Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story,” *Computer*, vol. 45, n.º 2, págs. 37-42, 2012. DOI: [10.1109/MC.2012.33](https://doi.org/10.1109/MC.2012.33).
- [3] M. T. Nygard, *Release It!: Design and Deploy Production-Ready Software*, 2.^a ed. Pragmatic Bookshelf, 2018, ISBN: 978-1-68050-239-8.